

Project 1 on Machine Learning

Regression analysis, resampling methods and gradient descent

Deadline October 5 (midnight), 2026

Data Analysis and Machine Learning FYS-STK3155/FYS4155
Department of Physics, University of Oslo, Norway

August 31, 2026

Preamble: note on writing reports, using reference material, AI and other tools

We want you to answer the three different projects by handing in reports written like a standard scientific/technical report. The links at <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/Projects> contain more information. There you can find examples of previous reports, the projects themselves, how we grade reports etc. How to write reports will also be discussed during the various lab sessions. Please do ask us if you are in doubt. In that folder you will find how we evaluate the projects. You will receive a feedback with a final score for each project.

When using codes and material from other sources, you should refer to these in the bibliography of your report, indicating wherefrom you for example got the code, whether this is from the lecture notes, software like **Scikit-Learn**, **JAX**, **TensorFlow**, **PyTorch** or other sources. These sources should always be cited correctly. How to cite some of the libraries is often indicated from their corresponding GitHub sites or websites; see for example how to cite Scikit-Learn at <https://scikit-learn.org/dev/about.html>.

We encourage you to use Large Language models (LLMs) like **ChatGPT**, **Claude** or similar in writing the report and in developing your code. In the report, you should after the conclusions, write a declaration on how you have used LLMs following the instructions at https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md. Failing to do so, leads to an automatic deduction of five (5) points out of the final score of 100 for the project. Remember that *you* are responsible for every line of code and every claim in the report: be prepared to explain and defend all of it. If you use LLMs, please take a look at the guidelines at <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/UsingLLMs> on how to use these tools efficiently.

If you would like to study other data sets, feel free to propose other sets. What we have proposed here are mere suggestions from our side. If you opt for another data set, consider using a set which has been studied in the scientific literature. This makes it easier for you to compare and analyze your results. Comparing with existing results from the scientific literature is also an essential element of the scientific discussion. The University of California at Irvine with its Machine Learning repository at <https://archive.ics.uci.edu/ml/index.php> is an excellent site to look up for examples and inspiration. [Kaggle.com](https://www.kaggle.com) is an equally interesting site. Feel free to explore these sites and/or ask us for inputs relevant to your discipline. When selecting other data sets, make sure these are sets used for regression problems (not classification).

Regression analysis, resampling methods and gradient descent

The main aim of this project is to study in more detail various regression methods, including Ordinary Least Squares (OLS) regression, Ridge regression and Lasso regression, together with the two resampling techniques (the bootstrap and cross-validation) that let us assess and select models, and the gradient descent methods that will be our optimizers for the rest of the course. In addition to the scientific part, in this course we want also to give you an experience in writing scientific reports.

We will study how to fit polynomials to a specific one-dimensional function (feel free to replace the suggested function with more complicated ones). We will use Runge's function (see https://en.wikipedia.org/wiki/Runge%27s_phenomenon for a discussion),

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1].$$

The project is organized so that it follows the weekly exercises and lectures of the first weeks of the course:

- Parts a) and b): OLS and Ridge regression with closed-form solutions (weeks 34–36; the exercise sets of weeks 35 and 36 can be reused directly).
- Parts c) and d): the bias-variance tradeoff studied with the bootstrap, and cross-validation (week 36; Chapter 2, Sections 2.9–2.12 of the lecture notes).
- Parts e) and f): replacing the closed-form solutions by gradient descent, with gradients computed both analytically and by automatic differentiation, and with momentum and adaptive learning rates (weeks 37–38; Chapter 4).
- Parts g) and h): Lasso regression, our first method without a closed-form solution, and stochastic gradient descent (chapter 4 of the lecture notes).
- Part i): the final comparison of OLS, Ridge and Lasso using cross-validation.

The analytical parts (in particular the derivation of the bias-variance decomposition in part c) belong in the theory section of your report. The material for solving the various exercises is covered by chapters 2-4 of the lecture notes and the lecture material for weeks 34-38.

Part a): Ordinary Least Squares (OLS) for the Runge function

We will generate our own data set for the above function $f(x)$ with $x \in [-1, 1]$, using either a uniform distribution or a fixed step size to set up the array of x values. You should also explore the addition of normally distributed noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ to the function; a noise level of $\sigma \approx 0.1$ is a good starting point, but study how your conclusions depend on it.

Write your own code (using for example the pseudoinverse function `pinv` from `numpy`, or the SVD directly) and perform a standard **ordinary least squares regression** analysis using polynomials in x up to order 15 or higher. Explore the dependence on the number of data points and the polynomial degree.

Evaluate the mean squared error (MSE)

$$\text{MSE}(\mathbf{y}, \tilde{\mathbf{y}}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2,$$

and the R^2 score function. If $\tilde{y}_i$ is the predicted value of the i -th sample and y_i is the corresponding true value, then the score R^2 is defined as

$$R^2(\mathbf{y}, \tilde{\mathbf{y}}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i.$$

Plot the resulting scores (MSE and R^2) as functions of the polynomial degree (here up to polynomial degree 15). Plot also the parameters θ as you increase the order of the polynomial. Comment your results.

Your code has to include a scaling/centering of the data (for example by subtracting the mean value), and a split of the data in training and test data. For the splitting you can either write your own code or use for example the function `train_test_split` provided by **Scikit-Learn** (make sure you have installed it). **You should present a critical discussion of why and how you have scaled/ or not scaled the data** (see Section 3.13 of the lecture notes and the Tuesday session of week 35 and the pertinent notebook for week 35).

It is normal in essentially all Machine Learning studies to split the data in a training set and a test set (eventually also an additional validation set). There is no explicit recipe for how much data should be included as training data and say test data. An accepted rule of thumb is to use approximately 2/3 to 4/5 of the data as training data.

You can easily reuse the solutions to your exercises from week 35 (the analytical derivatives of the cost function, your own OLS code, and the train/test analysis). See also the lecture slides from weeks 34 and 35 and Chapter 3 of the lecture notes. On scaling, we recommend reading the following section from the Scikit-Learn documentation:

https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html

. See also section 3.13 of the lecture notes.

Part b): Adding Ridge regression for the Runge function

Write your own code for the Ridge method, as done in the previous part. The lecture notes from weeks 35 and 36 contain more information, as well as your exercises from weeks 35 and 36. Feel free to reuse this material.

Perform the same analysis as you did in the previous part but now for different values of the penalty parameter λ . Compare and analyze your results with those obtained in part a) with the OLS method. Study the dependence on λ , and connect what you see to the shrinkage of the singular-value modes discussed in the lectures (Section 3.8 of the lecture notes).

Part c): The bias-variance tradeoff and the bootstrap

Our aim here is to study the bias-variance tradeoff by implementing the **bootstrap** resampling technique. **We will only use the simpler ordinary least squares here.**

Before you perform an analysis of the bias-variance tradeoff on your test data, make first a figure similar to Fig. 2.11 of Hastie, Tibshirani and Friedman. Figure 2.11 of this reference displays only the test and training MSEs. The test MSE can be used to indicate possible regions of low/high bias and variance. You will most likely not get an equally smooth curve! You will also need to increase the polynomial order and play around with the number of data points as well (see the exercise sets from week 35 and week 36).

With this result we move on to the bias-variance tradeoff analysis. Consider a data set $\mathcal{L}$ consisting of the data $\mathbf{X}_{\mathcal{L}} = \{(y_j, \mathbf{x}_j), j = 0, \dots, n-1\}$, and assume that the true data are generated from a noisy model

$$\mathbf{y} = f(\mathbf{x}) + \varepsilon,$$

where ε is normally distributed with mean zero and variance σ^2 . In our derivation of the ordinary least squares method we defined an approximation to the function f in terms of the parameters

$\boldsymbol{\theta}$ and the design matrix $\mathbf{X}$ which embody our model, that is $\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$. The parameters $\boldsymbol{\theta}$ are in turn found by optimizing the mean squared error via the cost function

$$C(\mathbf{X}, \boldsymbol{\theta}) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \mathbb{E} \left[(\mathbf{y} - \tilde{\mathbf{y}})^2 \right].$$

Here the expected value $\mathbb{E}$ is the sample value.

Show that you can rewrite this in terms of a term which contains the variance of the model itself (the so-called variance term), a term which measures the deviation from the true data and the mean value of the model (the bias term) and finally the variance of the noise. That is, show that

$$\mathbb{E} \left[(\mathbf{y} - \tilde{\mathbf{y}})^2 \right] = \text{Bias}^2[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] + \sigma^2,$$

with

$$\text{Bias}^2[\tilde{\mathbf{y}}] = \mathbb{E} \left[(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])^2 \right], \quad \text{var}[\tilde{\mathbf{y}}] = \mathbb{E} \left[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2 \right] = \frac{1}{n} \sum_i (\tilde{y}_i - \mathbb{E}[\tilde{\mathbf{y}}])^2.$$

Important note: since the function $f(x)$ is unknown, in order to be able to evaluate the bias we replace $f(x)$ in the expression for the bias with $\mathbf{y}$; discuss what this does to the measured bias term (it absorbs the noise variance σ^2). Explain what the three terms mean and discuss their interpretations, and state clearly over which randomness the expectation values are taken.

The answer to this exercise should be included in the theory part of the report. This derivation is also part of the weekly exercises of week 36, and it is Eq. (2.52) of Chapter 2 of the lecture notes.

Perform then a bias-variance analysis of the Runge function by studying the MSE value as function of the complexity of your model, using the **bootstrap** resampling method to estimate the expectation values over training sets. Discuss the bias and variance tradeoff as function of your model complexity (the degree of the polynomial) and the number of data points. You can follow the code examples in Sections 2.10 and 2.11 of the lecture notes and the Tuesday notebook of week 36 (`week36tuesday.ipynb`).

Part d): Cross-validation as a resampling technique

The aim here is to use another widely popular resampling technique, the so-called cross-validation method (Section 2.12 of the lecture notes and the Tuesday session of week 36). For Ridge and Lasso, see also section 3.14 of the lecture notes.

Use the k -fold cross-validation functionality of **Scikit-Learn** (`KFold` together with `cross_val_score`, or `cross_validate`) and evaluate again the MSE function resulting from the test folds, for the OLS analysis of parts a) and c) as a function of the polynomial degree, and for Ridge regression as a function of both the polynomial degree and λ . Try $k = 5$ and $k = 10$. Optionally, you can also write your own k -fold cross-validation code and check that it reproduces the Scikit-Learn results when the two use the same fold assignment (the Tuesday session of week 36 shows how).

Compare the MSE you get from cross-validation with the one you got from your **bootstrap** code in part c). Do the two methods point to the same model complexity? Comment and interpret your results. Make sure that any scaling or other preprocessing that learns from the data is performed *inside* the cross-validation loop (fitted on the training folds only), and explain why this matters.

Part e): Writing your own gradient descent code, with analytical gradients and automatic differentiation

Replace now the analytical expressions for the optimal parameters θ of OLS and Ridge regression with your own gradient descent code. In this part we focus only on the simplest gradient descent approach with a fixed learning rate η (see the lectures and exercises of week 37 and Sections 4.3–4.5 of the lecture notes),

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} C(\theta_k).$$

Compute the gradient of the cost function in two independent ways:

1. **Analytically**, from the expressions you derived in the week-35 exercises, that is $\nabla_{\theta} C = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$ for OLS and the corresponding expression with the extra term $2\lambda\theta$ for Ridge (check your normalization conventions).
2. By **automatic differentiation** (Section 4.14 of the lecture notes), using for example **JAX** (`jax.grad`) or **Autograd**. Write the cost function as an ordinary Python function of θ and let the library produce its gradient.

Verify that the two gradients agree to machine precision for OLS and Ridge, and use both in your gradient descent code. Discuss briefly why automatic differentiation is neither symbolic nor numerical (finite-difference) differentiation, and what its cost is relative to one evaluation of the cost function. A minimal example of the check is given in the notebook version of this project.

Study and compare your results from parts a) and b) with your gradient descent approach: do you converge to the closed-form solutions, and how many iterations does it take? Discuss in particular the role of the learning rate, and relate the largest learning rate for which the iteration converges to the largest eigenvalue of the Hessian matrix $\frac{2}{n} \mathbf{X}^T \mathbf{X}$ (plus $2\lambda \mathbf{I}$ for Ridge), see Section 4.5 of the lecture notes.

Part f): Including momentum and more advanced ways to update the learning rate

We keep our focus on OLS and Ridge regression and update our gradient descent code by including **momentum**, **AdaGrad**, **RMSprop** and **Adam** as methods for iteratively updating the learning rate (Sections 4.6 and 4.8–4.11 of the lecture notes; the lectures of weeks 37 and 38 contain several examples on how to implement these methods). You are free to use either the analytical or the automatically differentiated gradient here – the update rules do not care where the gradient comes from.

Discuss the results and compare the different methods applied to the one-dimensional Runge function, for example in terms of the number of iterations needed to reach a given accuracy relative to the closed-form solution, and in terms of their sensitivity to the choice of the (initial) learning rate.

Part g): Writing our own code for Lasso regression

Lasso regression (Section 3.9 of the lecture notes, and the lectures of weeks 35 and 36) represents our first encounter with a machine learning method which cannot be solved through analytical expressions (as in OLS and Ridge regression). Use the gradient descent methods you developed in parts e) and f) to solve the Lasso optimization problem.

The ℓ_1 penalty $\lambda \|\theta\|_1$ is not differentiable at $\theta_j = 0$. Discuss how you could handle this: what does your automatic differentiation tool return for the derivative of $|\theta|$ at zero, and is that a valid subgradient? Based on the gradient, write your own code for Lasso regression.

You can compare your results with the functionalities of **Scikit-Learn**; be careful with the different conventions for the penalty parameter (**alpha**) and the normalization of the cost function discussed in the week-35 and week-36 Tuesday sessions.

Discuss (critically) your results for the Runge function from OLS, Ridge and Lasso regression using the various gradient descent approaches.

Part h): Stochastic gradient descent

Our last gradient step is to include stochastic gradient descent (Section 4.7 of the lecture notes) using the same methods to update the learning rates as in parts e)–g). Split the training data into mini-batches, loop over epochs, and study the dependence on the mini-batch size, the number of epochs and the learning-rate schedule. Compare and discuss your results with and without stochastic gradient descent and give a critical assessment of the various methods – both in terms of the final accuracy relative to the closed-form solutions of parts a) and b), and in terms of computational cost.

Part i): Final analysis – OLS, Ridge and Lasso with cross-validation

Bring the pieces together. Using the cross-validation setup from part d), perform a final model selection for the Runge function with all three methods: OLS as a function of the polynomial degree, and Ridge and Lasso as functions of both the polynomial degree and λ . Present the results as a small number of well-chosen figures or tables, identify the best model according to the cross-validated test error, and discuss critically the strengths and weaknesses of the three methods for this problem. Which of the terms in the bias-variance decomposition of part c) does each method attack, and does your numerical evidence support that reading?

Background literature

- The lecture notes of the course, in particular Chapter 2 (Sections 2.9–2.12 on training and test error, the bias-variance tradeoff, the bootstrap and cross-validation), Chapter 3 (linear regression, Ridge, Lasso, scaling) and Chapter 4 (gradient descent methods and automatic differentiation).
- For a discussion and derivation of the variances and mean squared errors using linear regression, see the [Lecture notes on ridge regression by Wessel N. van Wieringen](#).
- The textbook of [Trevor Hastie, Robert Tibshirani, Jerome H. Friedman, The Elements of Statistical Learning, Springer](#); chapters 3 and 7 are the most relevant ones for the analysis here.
- On automatic differentiation: [Baydin et al., Automatic differentiation in machine learning: a survey, JMLR 18 \(2018\)](#), and the [JAX documentation](#).

Introduction to numerical projects

Here follows a brief recipe and recommendation on how to answer the various questions when preparing your answers.

- Give a short description of the nature of the problem and the eventual numerical methods you have used.
- Describe the algorithm you have used and/or developed. Here you may find it convenient to use pseudocoding. In many cases you can describe the algorithm in the program itself.
- Include the source code of your program. Comment your program properly. You should have the code at your GitHub/GitLab link. You can also place the code in an appendix of your report.

- If possible, try to find analytic solutions, or known limits in order to test your program when developing the code. In this project the closed-form OLS and Ridge solutions are exactly such tests for your gradient descent codes.
- Include your results either in figure form or in a table. Remember to label your results. All tables and figures should have relevant captions and labels on the axes.
- Try to evaluate the reliability and numerical stability/precision of your results. If possible, include a qualitative and/or quantitative discussion of the numerical stability, eventual loss of precision etc.
- Try to give an interpretation of your results in your answers to the problems.
- Critique: if possible include your comments and reflections about the exercise, whether you felt you learnt something, ideas for improvements and other thoughts you've made when solving the exercise. We wish to keep this course at the interactive level and your comments can help us improve it.
- Try to establish a practice where you log your work at the computer lab. You may find such a logbook very handy at later stages in your work, especially when you don't properly remember what a previous test version of your program did. Here you could also record the time spent on solving the exercise, various algorithms you may have tested or other topics which you feel worthy of mentioning.

Format for electronic delivery of report and programs

The preferred format for the report is a PDF file. You can also use DOC or postscript formats or an IPython notebook file. As programming language we prefer that you choose between C/C++, Fortran2008, Julia or Python. The following prescription should be followed when preparing the report:

- Use Canvas to hand in your projects, log in at <https://www.uio.no/english/services/it/education/canvas/> with your normal UiO username and password.
- Upload **only** the report file or the link to your GitHub/GitLab or similar type of repository! For the source code file(s) you have developed please provide us with your link to your GitHub/GitLab or similar domain. The report file should include all of your discussions and a list of the codes you have developed. Do not include library files which are available at the course homepage, unless you have made specific changes to them.
- In your GitHub/GitLab or similar repository, please include a folder which contains selected results. These can be in the form of output from your code for a selected set of runs and input parameters.

Finally, we encourage you to collaborate. Optimal working groups consist of 2–3 students. You can then hand in a common report.

Software and needed installations

If you have Python installed (we recommend Python 3) and you feel pretty familiar with installing different packages, we recommend that you install the following Python packages via `pip` as

```
pip install numpy scipy matplotlib ipython jupyter scikit-learn pandas
pip install jax jaxlib
```

(`autograd` is a lighter alternative to JAX for the automatic differentiation in parts e)–g)). For Python 3, replace `pip` with `pip3` where needed.

For OSX users we recommend also, after having installed Xcode, to install `brew`. Brew allows for a seamless installation of additional software via for example `brew install python3`. For Linux

users, with its variety of distributions like for example the widely popular Ubuntu distribution, you can use `pip` as well and simply install Python as `sudo apt-get install python3` etc.

If you don't want to install various Python packages with their dependencies separately, we recommend [Anaconda](#), an open source distribution of the Python and R programming languages for large-scale data processing, predictive analytics, and scientific computing, that aims to simplify package management and deployment. Package versions are managed by the package management system `conda`.

Popular software packages written in Python for ML are [Scikit-learn](#), [JAX](#), [TensorFlow](#), [PyTorch](#) and [Keras](#). These are all freely available at their respective GitHub sites. They encompass communities of developers in the thousands or more, and the number of code developers and contributors keeps increasing.